

LAPORAN PRAKTIKUM INTERNET OF THINGS

Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)



Oleh:

NAMA : Albertus Satriyo Nugroho
NPM 25782002
KELAS : 3A
MATKUL : Internet of Things (IOT)

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T. Agus
Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET JURUSAN
TEKNOLOGI INFORMASI POLITEKNIK NEGERI
LAMPUNG 2026

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?

JAWAB:

Sebelum dicabut: Callback `OnDataSent` dapat respon `sendStatus == 0` (sukses). Di monitor bakal keluar log kayak "Berhasil Diterima", waktu NodeMCU 2 dicabut lalu dinyalakan kembali pengirimannya, status buat MAC address NodeMCU 2 langsung berubah jadi non-zero (`ESP_NOW_SEND_FAIL`). Di log bakalan langsung ganti jadi "Gagal (Tidak Terjangkau)".

NodeMCU 1 (penerima satunya): Karena masih tancap gas dan dapat daya, log kirim ke MAC address NodeMCU 1 bakal tetap "Berhasil Diterima". Jadi failure-nya benar-benar spesifik ke node yang mati aja.

Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

JAWAB:

ESP-NOW bersifat *connectionless* tanpa koneksi TCP, protokol ini bekerja langsung di Layer 2 (MAC Layer) Wi-Fi 802.11. Setiap Controller mengirim data ke MAC address tujuan, chip Wi-Fi pengirim bakal nunggu balasan bingkai ACK dari chip penerima secara *hardware* dalam hitungan mikrodetik.

Pas NodeMCU penerima dicabut dayanya, chip radionya mati sehingga bingkai ACK tidak terkirim. Hardware pengirim yang mengalami *ACK Timeout* bakal nyoba retransmisi otomatis (*hardware retry*) beberapa kali. Kalau tetap tidak ada respon, MAC Layer melapor ke *driver* ESP-NOW untuk memicu fungsi *callback* `OnDataSent` dengan status `ESP_NOW_SEND_FAIL`, lalu mencetak log Gagal (Tidak Terjangkau).

Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4)

Jalankan jaringan `painlessMesh` pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

Tekan tombol fisik RST atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh `callback changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

JAWAB:

butuh waktu sekitar 3 sampai 5 detik buat menyadari ada node yang hilang. Di Serial Monitor bakal keluar log perubahan topologi kalau node tersebut udah keluar dari rantai mesh.

Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

JAWAB:

Pas daya board disambungkan lagi, butuh waktu sekitar 5 sampai 10 detik sampai node itu terdaftar ulang secara otonom. Waktu ini mencakup proses *booting* board, *scanning* channel Wi-Fi, pendaftaran IP via DHCP internal mesh, sampai akhirnya muncul log

Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai Pengawas Kondisi Suhu Kritis: program HANYA mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai di atas 32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).

Jika suhu bernilai normal ($\leq 32.0\text{ }^{\circ}\text{C}$), program cukup mencetak status singkat satu baris: `[NORMAL]` Telemetri dari Node `<chipId>` aman.

Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

JAWAB:

```
#include <ArduinoJson.h> // Pastikan library ArduinoJson sudah ter-include

void receivedCallback(uint32_t from, String &msg) {
  // 1. Alokasikan dokumen JSON untuk parsing
  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, msg);

  // Cek jika parsing JSON gagal
  if (error) {
    Serial.print("Gagal parsing JSON dari Node ");
    Serial.print(from);
    Serial.print(": ");
    Serial.println(error.f_str());
    return;
  }
}
```

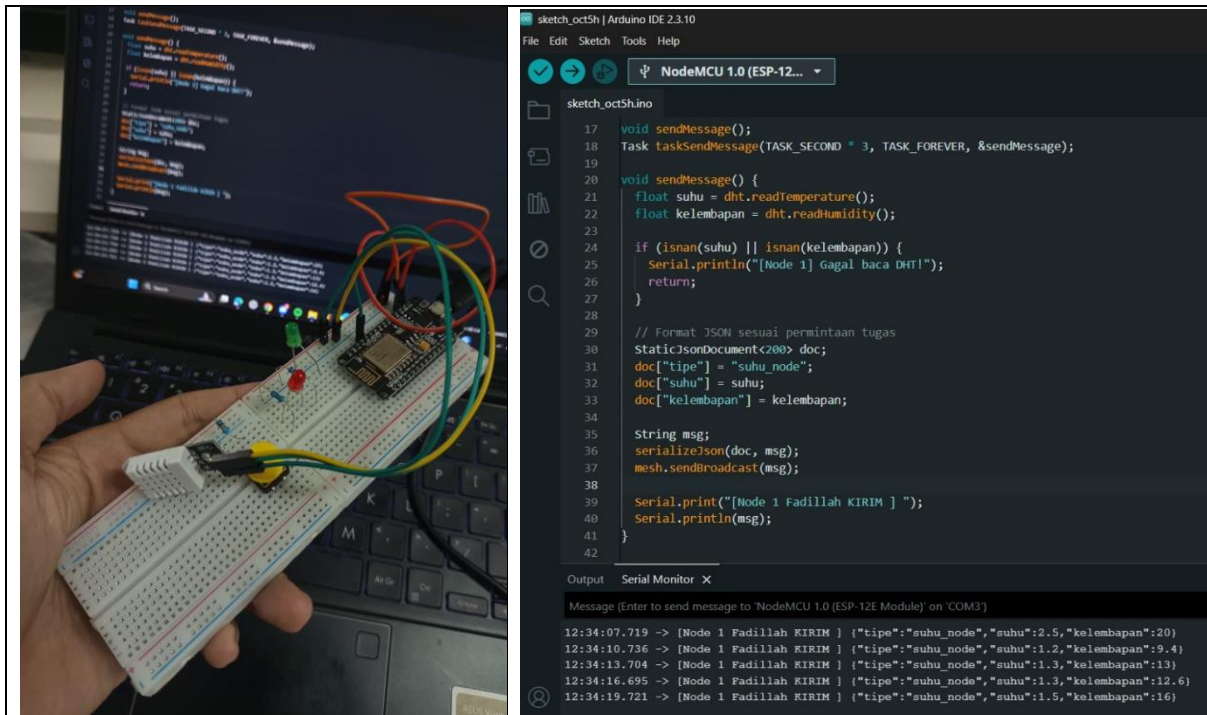
```
// 2. Ambil data suhu dari payload JSON
float temp = doc["temp"]; // Asumsi key untuk suhu di JSON adalah "temp"

// 3. Logika Pemfilteran Berdasarkan Ambang Batas 32.0 °C
if (temp > 32.0) {
    // KONDISI KRITIS: Cetak kartu telemetry penuh
    Serial.println("=====");
    Serial.println("    [PERINGATAN SUHU KRITIS]    ");
    Serial.println("=====");
    Serial.printf("Dari Node ID : %u\n", from);
    Serial.printf("Suhu      : %.2f °C\n", temp);
    if (doc.containsKey("humidity")) {
        Serial.printf("Kelembapan  : %.2f %%\n", doc["humidity"].as<float>());
    }
    Serial.println("Status      : MELEBIHI AMBANG BATAS!");
    Serial.println("=====\\n");
} else {
    // KONDISI NORMAL: Cetak status singkat satu baris
    Serial.printf("[NORMAL] Telemetry dari Node %u aman.\\n", from);
}
}
```

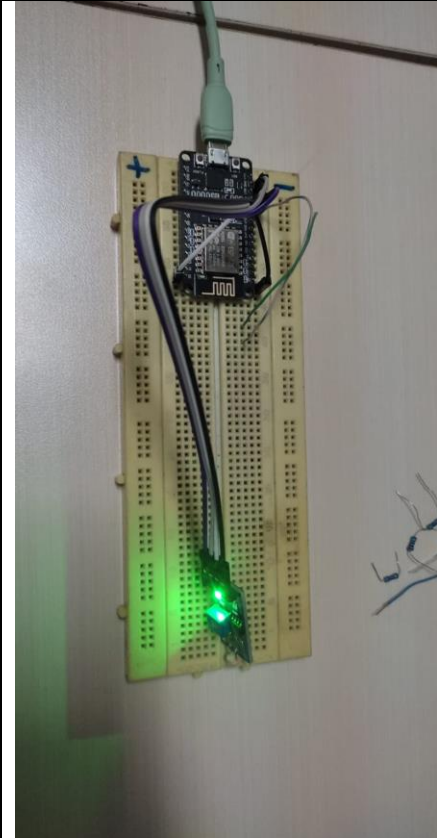
5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh

HASIL TUGAS

A. DHT11



B. LDR



```

sketch_oct5b.ino
29   Serial.println(msg);
30 }
31
32 void setup() {
33   Serial.begin(115200);
34
35   mesh.setDebugMsgTypes(ERROR | STARTUP);
36   mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
37
38   userScheduler.addTask(taskSendMessage);
39   taskSendMessage.enable();
40 }
41
42 void loop() {
43   mesh.update();
44 }
  
```

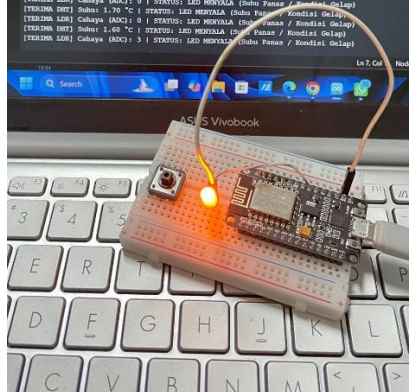
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

```

[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":1}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":1}
[Node 2 KIRIM] {"tipe":"cahaya_node","adc":0}
  
```

C. LED



```

sketch_oct5a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
sketch_oct5a.ino
1 #include <painlessMesh.h>
2 #include <ArduinoJson.h>
3
4 #define MESH_PREFIX "LabIoTMesh"
5 #define MESH_PASSWORD "iotmeshpassword"
6 #define MESH_PORT 5555
7
8 #define LEDPIN 12 // Pin D6 (GPIO 12)
9
10 Scheduler userScheduler;
11 painlessMesh mesh;
12
13 // Variabel untuk menyimpan pembacaan terakhir
14 float suhuTerakhir = 0.0;
15 int adcTerakhir = 1024; // Default nilai tinggi (terang)
16
  
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```

[TERIMA DHT] Suhu: 2.40 °C | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA LDR] Cahaya (ADC): 1 | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA DHT] Suhu: 2.40 °C | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA LDR] Cahaya (ADC): 0 | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA DHT] Suhu: 2.30 °C | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA LDR] Cahaya (ADC): 0 | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA DHT] Suhu: 2.30 °C | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
[TERIMA LDR] Cahaya (ADC): 3 | STATUS: LED MENYALA (Suhu Panas / Kondisi Gelap)
  
```